🧭 Personal DeepAgent: Codebase Architecture & Learning Plan

Welcome to the structured learning roadmap for **Personal DeepAgent**. This guide breaks down the entire system into 6 digestible learning modules, moving from the foundational data layer up to the autonomous agent reasoning graph and the interactive live arena UI.

🧬 High-Level System Architecture

Before diving into individual files, here is how the data and execution flow through the codebase:

```
flowchart TD
    subgraph Ingestion ["1. INGESTION PIPELINE (src/skills/ingestion)"]
        PDF["Research PDF / arXiv URL"] ⟶ Parser["PDF Parser (parser.py)"]
        Parser ⟶ Extractor["Metadata Extractor (extractor.py)"]
        Parser ⟶ Chunker["Header-Aware Chunker (chunker.py)"]
        Chunker ⟶ Evaluator["Needle Recall Evaluator (evaluator.py)"]
        Evaluator ⟶ Pipeline["Ingestion Orchestrator (pipeline.py)"]
    end

    subgraph Storage ["2. DUAL-STORAGE ENGINE (src/skills & src/utils)"]
        Pipeline ⟶|"Structured Meta"| DuckDB[("DuckDB Catalog (data/catalog.csv)")]
        Pipeline ⟶|"Text Embeddings"| Chroma[("ChromaDB Vector Store (data/chroma_db)")]
    end

    subgraph Retrieval ["3. TWO-STAGE HYBRID RETRIEVAL (src/skills/retriever_skill.py)"]
        Query["User Query"] ⟶ BM25["Sparse Okapi BM25 Keyword Search"]
        Query ⟶ Dense["Dense Chroma Cosine Similarity"]
        BM25 ⟶ RRF["Reciprocal Rank Fusion (RRF)"]
        Dense ⟶ RRF
        RRF ⟶ FlashRank["FlashRank CPU Cross-Encoder Reranker"]
        FlashRank ⟶ TopChunks["Top Filtered Chunks (No Candidate Crowding)"]
    end

    subgraph AgentLoop ["4. AUTONOMOUS AGENT REASONING (src/agent.py & src/prompt.py)"]
        LLM["DeepSeek / LLM Engine"] ⟷ Graph["LangGraph ReAct Execution Loop"]
        Graph ⟶|"Step 1"| Planner["To-Do Planner (todo_write)"]
        Graph ⟶|"Step 2"| SQLTool["DuckDB Catalog Check (catalog_query)"]
        Graph ⟶|"Step 3"| TopChunks
        Graph ⟶|"Step 4"| Scratchpad["Virtual Filesystem Workspace (write_file)"]
        Graph ⟶|"Step 5"| Sanitizer["Response Sanitizer & Table Grounding (sanitizer.py)"]
    end

    subgraph Interface ["5. INTERACTION & ARENA (src/api.py & src/templates)"]
        UserUI["Web Browser (http://localhost:8000)"] ⟷ FastAPIServer["FastAPI Server (src
        FastAPIServer ⟶ AgentLoop
        FastAPIServer ⟶ NaiveRAG["Naive RAG Baseline (Blind Single-Shot)"]
        FastAPIServer ⟶ UI["Live Arena & Chunk Studio (viewer.html)"]
    end
```

🗓️ The 5-Phase Learning Roadmap

Follow this sequence to build a complete mental model of the codebase:

```
flowchart LR
    P1["Phase 1: Ingestion & Chunker"] ⟶ P2["Phase 2: Storage & Hybrid Search"]
    P2 ⟶ P3["Phase 3: The Autonomous ReAct Agent"]
    P3 ⟶ P4["Phase 4: Live Arena & FastAPI Backend"]
    P4 ⟶ P5["Phase 5: Testing & CI/CD Pipeline"]
```

📦 Phase 1: Ingestion, Chunking & Needle Evaluation

> **Goal**: Understand how raw PDFs are converted into structured, high-recall knowledge fragments.

Key Files to Read:

1. `src/skills/ingestion/schemas.py`
   - Defines Pydantic schemas: `DocumentMetadata`, `Chunk`, and `IngestionReport`.
   - Pay attention to the fields: `header_path`, `word_count`, and `recall_score`.
2. `src/skills/ingestion/parser.py`
   - Uses PyMuPDF (`fitz`) to extract clean text and detect markdown headers (`#`, `##`, `###`).
3. `src/skills/ingestion/chunker.py`
   - **Core Innovation**: Why fixed-size chunking hurts academic papers.
   - See how `HeaderAwareChunker` preserves section context (e.g. `/INTRODUCTION`, `/METHOD/3.1 Prompting`) so chunks retain provenance.
4. `src/skills/ingestion/evaluator.py`
   - Implements automated "Needle-in-a-Haystack" testing to calculate a recall score before committing chunks.
5. `src/skills/ingestion/pipeline.py`
   - Glues parsing, chunking, evaluating, and writing into one unified flow (`ingest_pdf_document`).

💡 **Hands-On Exercise for Phase 1:**

Run a manual ingestion from the CLI to observe the pipeline in terminal:

```
uv run python scripts/ingest_paper.py --source data/papers/2210.03629v3.pdf --threshold 0.8
```

🔍 Phase 2: Dual-Storage & Two-Stage Hybrid Retrieval

> **Goal**: Master how structured SQL data and unstructured vector data complement each other, and how FlashRank eliminates "Candidate Crowding".

Key Files to Read:

1. `src/skills/catalog_skill.py`
   - Exposes DuckDB querying over `data/catalog.csv`.
   - Notice why an agent needs SQL: LLMs struggle with exact paper tallies and filtering by year/author in raw vector search.
2. `src/utils/chroma.py`
   - Initializes ChromaDB with persistent storage at `data/chroma_db`.

3. `src/skills/retriever_skill.py`
    - **The 2-Stage Retrieval Architecture**:
        - **Stage 1 (Fusion)**: Runs ChromaDB dense vector search + BM25 sparse keyword search, combined via **Reciprocal Rank Fusion (RRF)**: $$RRF_Score(d) = \sum_{m \in M} \frac{1}{60 + r_m(d)}$$
        - **Stage 2 (Cross-Encoder)**: Passes the top 20 fused candidates through `FlashRank` (`ms-marco-TinyBERT-L-2-v2`) on CPU to produce the final top-4 highest density chunks.

💡 **Hands-On Exercise for Phase 2:**

Test semantic retrieval and inspect raw chunks directly:

```
uv run python -c "from src.skills.retriever_skill import retrieve_knowledge_chunks; chunks =
```

🧠 Phase 3: The Autonomous ReAct Agent & System Protocols

> **Goal**: Understand how LangGraph, DeepSeek, and prompt engineering create true autonomous decision-making.

Key Files to Read:

1. `src/prompt.py`
    - The brain of the agent. Study the 3 Operational Protocols:
        - **Protocol 1**: Concept & Definition Queries (Active planning + single-paper verification).
        - **Protocol 2**: Parallel 2-Hop Comparative Research (Parallel multi-paper probes, scratchpad drafting, comparative matrices).
        - **Protocol 3**: Pure Counts & Listings (Strictly DuckDB SQL only).
2. `src/agent.py`
    - Assembles the agent using `deepagents.create_deep_agent`.
    - Binds the tool suite: `catalog_query`, `knowledge_retriever`, `todo_write`, `todo_read`, `write_file`, `read_file`.
    - Implements `run_agent_turn` to execute the graph and capture messages.
3. `src/utils/sanitizer.py`
    - Post-processing pipeline invariants:
        - Sanitizes Markdown table cell pipes (`|` $\rightarrow$ `–`) to prevent broken table columns.
        - Restores 100% accurate, verbatim arXiv URLs in the References section.

💡 **Hands-On Exercise for Phase 3:**

Run a standalone agent turn from Python:

```
uv run python -c "from src.agent import run_agent_turn; res = run_agent_turn('what is ReAct')
```

⚔️ Phase 4: Head-to-Head Arena & FastAPI Web Studio

> **Goal**: Learn how the telemetry benchmark compares Naive RAG vs DeepAgent side-by-side.

Key Files to Read:

1. `scripts/benchmark_comparison.py`
   - Defines `run_naive_rag`: Single vector query $\rightarrow$ prompt stuffing $\rightarrow$ unverified generation.
   - Defines `run_deepagent_benchmark`: Invokes the agent, tracks message history, and constructs the step-by-step procedural trace (`execution_steps`).
2. `src/api.py`
   - Serves endpoints:
     - `POST /api/arena`: Runs or retrieves comparison results.
     - `GET /api/documents`: Catalog list with chunk counts.
     - `POST /api/ingest`: Triggers live paper ingestion.
3. `src/templates/viewer.html`
   - Vanilla JS + Tailwind CSS dark minimalist UI.
   - Look at `renderExecutionSteps`: Renders the collapsible `<details>` thinking process accordion.
   - Look at `streamMarkdown`: Progressive typewriter token streaming with auto-scroll.

💡 Hands-On Exercise for Phase 4:

Start the server and inspect the API schema in Swagger UI:

```
uv run python scripts/run_server.py
```

Open `http://localhost:8000/docs` in your browser.

🧪 Phase 5: Verification, Unit Testing & CI/CD

> **Goal**: Understand how automated test coverage protects every layer of the architecture.

Key Files to Read:

1. `tests/test_agent_assembly.py`: Tests agent graph compilation and tool binding.

2. `tests/test_retriever_skill.py`: Tests BM25, Chroma, and FlashRank reranking.
3. `tests/test_sanitizer.py`: Tests markdown pipe sanitization.
4. `tests/test_api.py`: FastAPI endpoint testing using `TestClient`.
5. `.github/workflows/ci.yml`: GitHub Actions workflow running Ruff linting and Pytest on every push.

💡 **Hands-On Exercise for Phase 5:**

Run the test suite locally:

```
uv run pytest -v
```

🗺️ Codebase File Reference Directory

Directory / File	Core Responsibility	Technologies Used
`src/agent.py`	Agent compilation, tool registration, turn runner	`langgraph`, `deepagents`, `langchain-core`
`src/prompt.py`	System prompt, operational protocols	Prompt Engineering
`src/skills/retriever_skill.py`	Hybrid dense+sparse search & cross-encoder	`chromadb`, `rank-bm25`, `flashrank`
`src/skills/catalog_skill.py`	SQL querying over metadata catalog	`duckdb`, `pandas`
`src/skills/ingestion/`	PDF parsing, chunking & recall evaluation	`PyMuPDF (fitz)`, `pydantic`
`src/utils/sanitizer.py`	Table pipe repair & URL preservation	Regex, string parsing
`src/api.py`	Web endpoints, Arena API, Ingestion API	`fastapi`, `uvicorn`
`src/templates/viewer.html`	Interactive Live Arena & Knowledge Studio UI	HTML5, Tailwind, `marked.js`, Vanilla JS
`scripts/benchmark_comparison.py`	Head-to-Head Naive RAG vs DeepAgent benchmark	`pydantic`, `tiktoken`

🎯 Recommended Next Step

Start with **Phase 1** and open `src/skills/ingestion/chunker.py` to see how academic papers are cleanly chopped into structured markdown sections before being stored!